

rvTTS: Near-Real-Time Hybrid Neural Speech Synthesis on a RISC-V Microcontroller

Armstrong Subero

Independent Researcher, Trinidad and Tobago

Abstract - We present rvTTS, a neural text-to-speech pipeline deployed on a RISC-V microcontroller and measured on physical silicon. A rule-based text frontend, a 198,136-parameter prosody model and a 403,604-parameter dilated convolutional acoustic model emit the 20-dimensional LPCNet feature vector, which is the interface the IETF Deep Redundancy extension for Opus [9] specifies and deliberately leaves non-normative. Two vocoders consume it. The first is a portable FARGAN reimplementation in three C99 files with no Opus build dependency, occupying 8132 bytes of text on RV32IMAC, with the weight blob reduced from 4.0 MB to 848 KB. The second is a classical LPC synthesiser driven by the identical features, carrying no weights, occupying 5012 bytes and costing 2.01 MMAC per second of audio against FARGAN's 304.7. Four measured improvements take it to from 2.414 to 3.114 SCOREQ and a frontend word identifier raises the complete pipeline to 3.172 against FARGAN's 3.744. On a CH32H417 dual core part at 150 and 300 MHz the LPC configuration renders typed text to audio at a real-time factor of 1.24, reduced from 85.3 through five measured optimisations. Two of those returned far less than instruction counts predicted, and the largest single gain came from reordering loops so that convolution weights are staged in RAM once per window rather than read from flash once per frame, a change that alters no arithmetic and is bit-identical to the code it replaces.

Index Terms - embedded systems, fixed-point inference, linear predictive coding, microcontrollers, on-device inference, quantisation, RISC-V, text-to-speech

I. INTRODUCTION

Speech synthesis on microcontrollers has remained largely unchanged for four decades. Software Automatic Mouth, originally developed for the Commodore 64 in 1982, and its derivatives remain widely used for microcontroller class text-to-speech. Such systems use formant synthesis with rule-based parameter tables, producing intelligible but distinctly robotic speech in under 20KB of program memory. That gap matters, because embedded voice applications are expanding into medical devices, industrial sensors, assistive technology and educational platforms, and those applications currently choose between cloud-dependent neural synthesis, which is high

quality but requires connectivity, and local formant synthesis, which is low quality and fully offline.

Neural vocoders have narrowed the quality gap on larger hardware. FARGAN [1] synthesises high-quality speech at 600 MFLOPS using pitch-prediction-based autoregression, and exists because of a specific application, the packet-loss concealment extension for Opus known as Deep Redundancy [8], [9]. DRED reconstructs missing audio from acoustic features when packets are lost, and those features are 18 Bark-frequency cepstral coefficients, a pitch period and a pitch correlation, computed on 20 ms windows at a 10 ms interval.

Two facts about that specification frame this paper. First, the acoustic feature set our pipeline emits is identical to DRED's, so any vocoder built for one works for the other. Second, the draft explicitly leaves the feature decoder normative while leaving the vocoder unspecified, which provides test vectors and a comparison tool rather than mandating an implementation. That clause is what allows an alternative vocoder to be substituted, and a vocoder cheap enough for a microcontroller is exactly the kind of alternative it permits.

This work makes four contributions.

1. We characterise int8 inference on RV32IMAC and RV32IMAFIC by exact retired instruction count rather than by scaled host timing, and show that the substitution of an int32 accumulator for a float32 one in the int8 matrix-vector kernel is worth a factor of 32 on a core without a floating-point unit while remaining bit-exact.
2. We present a portable FARGAN implementation in three C99 files with no libm and no Opus build dependency.
3. We show that a weightless classical vocoder driven by the same features reaches 3.172 SCOREQ against FARGAN's 3.744 for 0.7 percent of the arithmetic.
4. We report measured execution on physical silicon, where the analysis above overpredicted two of the five optimisations by large margins because an instruction count cannot see a weight fetch, and where the single largest gain came from a loop reordering that changes no arithmetic at all.

II. SYSTEM OVERVIEW

A. Pipeline and Feature Interface

The pipeline has three stages. A rule-based frontend converts text to phonemes and per-frame durations using a pronunciation dictionary. A prosody model predicts energy, pitch period and voicing from the phoneme sequence. An acoustic model predicts the remaining 17 cepstral dimensions conditioned on the prosody output. A vocoder converts the resulting 20-dimensional feature stream to 16 kHz PCM.

B. Acoustic Model Division

The division into two acoustic stages follows from a substitution experiment. Using ground truth features from LJSpeech through FARGAN as a ceiling at 1.881 PESQ, we replaced individual dimensions with model predictions and measured the loss. Replacing only the pitch period and voicing dimensions costs 0.282, while replacing the first six cepstral coefficients costs 0.820 and a constant pitch with otherwise perfect spectral coefficients yields only 1.122. Pitch period and voicing therefore dominate, which is consistent with FARGAN’s architecture, in which the pitch period drives a long-term prediction feedback loop that generates the harmonic structure of voiced speech. A dedicated model for the high-sensitivity features and a separate spectral model conditioned on its output is the architecture that follows.

The prosody model is a bidirectional GRU with 8-dimensional word embeddings, chosen because prosodic phenomena such as sentence-final declination and pre-boundary lengthening require future context. It predicts mean and log-variance for each output dimension, trained with Gaussian negative log-likelihood, and the deterministic mean is used at inference. The acoustic model is a purely dilated convolutional network with no recurrence, giving parallelisable computation and a receptive field of plus or minus 34 frames.

C. Target Hardware

The target is the WCH CH32H417, a dual core RISC-V system on chip with a QingKe V3F at 150 MHz and a V5F at 300 MHz, both RV32IMAFC with a single-precision floating-point unit, 896 KB of SRAM and 960 KB of flash arranged as two banks. A microarchitectural detail matters for inference. The V5F floating-point unit runs add at the core frequency divided by two, multiply at divided by three and fused multiply-accumulate at divided by four, so floating-point arithmetic is not free even where hardware support exists.

D. An Interaction Between Activation and Pitch Correction

An unexpected interaction between two implementation choices is worth recording here because it recurs as a theme. Replacing FARGAN’s rational polynomial activation approximations with 256-entry lookup tables and linear interpolation reduces the cost from roughly 8 floating-point operations per evaluation to 2, at a maximum absolute error of 0.000376 for tanh against 0.000060 for the polynomial. Both errors sit well below the noise floor of int8 weight quantisation, so the table was adopted for speed and expected to cost a little

quality. Table I shows that it does not, and that the activation approximation and the pitch correction mode interact.

TABLE I
ACTIVATION AND PITCH CORRECTION VARIANTS

Activation	Pitch correction	PESQ
Polynomial	Mean shift	1.745
Polynomial	Full normalise plus clamp	1.437
Lookup table	Mean shift	1.335
Lookup table	Full normalise	1.876
Lookup table	Full normalise plus clamp	1.911

Neither choice dominates on its own. The table is the worst configuration under mean shift and the best under variance normalisation, so the effect is an interaction rather than a property of either. We hypothesise that the table’s mild smoothing suits FARGAN’s autoregressive synthesis, because gate activations transition less abruptly between saturation regions and the quantisation of activation values onto a fixed grid acts as implicit regularisation in a feedback loop that runs for hundreds of sequential subframes. We have not tested either mechanism, and both are testable with the instrumentation described in Section VII.

III. INSTRUCTION-EXACT CHARACTERISATION OF RISC-V INFERENCE

A. Method

Cross architecture timing comparisons on host hardware are unreliable for predicting microcontroller behaviour. We instead compile the actual kernels for the target instruction set and count retired instructions under emulation.

The primary measurements were obtained using QEMU 8.2.2, package version 1:8.2.2+ds-0ubuntu1.16, running `qemu-system-riscv32` on the virt machine with `-cpu rv32`, with executed instructions counted by reading the minstret counter through `csrr` immediately before and after the kernel under test. The counter is only meaningful under `-icount shift=0`, and confirming that is a check on the harness rather than on the code. A loop of exactly 200,001 instructions, bracketed by the two counter reads, returns 200,003 under `-icount shift=0` and returns the same value on every run. Without the flag the same binary returns a non-deterministic value, observed between roughly 137,000 and 724,000 across repeated runs, because QEMU then advances the counter against host execution rather than against guest instructions. Every count reported below therefore uses the flag.

The figures are exact rather than statistical, so each is a single run and repeats bit for bit. Table II gives the result for the inner loops that dominate inference, measured on a 128 by 128 matrix-vector product, which is 16,384 multiply-accumulates.

TABLE II
INSTRUCTIONS PER MULTIPLY-ACCUMULATE BY KERNEL AND TARGET

Kernel	rv32imac	rv32imafc
int8 weights, float32 activations	229.2	7.06
int8 weights, int32 accumulator	7.06	7.06
tanh, 5-term polynomial and a divide	1172	11

B. Kernel Costs

The first row is the cost of the obvious implementation, in which int8 weights are widened to float and accumulated in float. On a core without a floating-point unit every one of those operations enters a soft-float routine. The second row is the same network with the accumulator changed to int32 and the scale applied after the dot product. That substitution is worth a factor of 32, and it is bit-exact rather than approximate, so a single numeric path serves both an FPU-bearing and an FPU-less target optimally. The presence of a floating-point unit stops being an architectural fork.

The third row is why activation tables matter on the same class of core. A single tanh through a polynomial and a divide costs 1172 instructions without hardware float and 11 with it, so the transcendental support code rather than the multiply-accumulates dominates an FPU-less build.

C. Independent Reproduction

Because the conclusion above rests on one toolchain, we repeated the measurement on a second. The figures in Table II were obtained with riscv-none-elf-gcc 15.2.0. The reproduction used riscv64-unknown-elf-gcc 13.2.0 at -O2 with link-time optimisation disabled, and qemu-riscv32 8.2.2 in user mode with -d exec,nochain -one-insn-per-tb, which emits one trace record per executed instruction.

It also uses a different technique. Rather than reading a counter around the kernel, it subtracts the whole-program instruction count of two otherwise identical binaries, one whose entry point calls the kernel and one whose entry point calls an empty function of the same signature, so that startup and initialisation cancel exactly. Two details matter for anyone repeating it. The kernel must be marked `noinline` or it is folded into its caller, and its outputs must be declared `volatile`, without which the compiler removes the loop as dead code and both binaries report the same count. Table III gives both sets of figures.

TABLE III
INDEPENDENT REPRODUCTION OF THE KERNEL COSTS

Kernel	Table II	Reproduction
int8 weights, int32 accumulator, rv32imac	7.06	7.07
int8 weights, float32 activations, rv32imac	229.2	228.88
int8 weights, int32 accumulator, rv32imafc	7.06	7.07
int8 weights, float32 activations, rv32imafc	7.06	7.08

Every figure agrees to within a third of one percent, and the gain from the accumulator substitution on the FPU-less core measures 32.4 against the 32 above. Two compiler generations and two independent measurement paths give the same answer, one counting architectural retirement through `minstret` under system emulation and the other counting executed instructions by tracing under user-mode emulation. That the two agree establishes the effect as a property of the instruction set rather than of one code generator or one counting method.

D. Consequences for Vocoder Choice

At 7.06 instructions per multiply-accumulate and 0.8 instructions per cycle, FARGAN's 304.7 MMAC per second of audio requires roughly 2.7 GHz to reach real time, which no part in this class provides. The LPC vocoder described in Section V requires 2.01 MMAC per second of audio, a factor of 152 less, and this is the reason the deployed configuration uses it.

IV. A PORTABLE FARGAN IMPLEMENTATION

A. Footprint

Building FARGAN from the Opus tree requires the Opus build system, its DSP library and `libm`. We reimplemented the inference graph as three files depending on nothing but `stdint.h` and `string.h`, with a single compile-time flag selecting float32 or int16 fixed-point activations. Table IV gives the footprint.

TABLE IV
PORTABLE FARGAN FOOTPRINT BY TARGET

Build	text	rodata	State
rv32imac, no FPU	8132 B	1030 B	2800 B
rv32imafc, float	6516 B	1030 B	4544 B

Neither build links `libm`. The only external symbols are `memcpy`, `memmove` and `memset`, which every embedded toolchain supplies.

B. Weight Blob Reduction

The weight blob as generated by the Opus source compiles to 4.0 MB. Cross-referencing the arrays shows that 3.09 MB of that is float copies of int8 layers already present, so removing the duplicates takes the blob to 848 KB with no change in output.

C. Read Traffic and Placement

Read traffic across the network differs by two orders of magnitude, which suggests a placement strategy rather than a single blob. Table V gives the measured traffic by group.

TABLE V
FARGAN WEIGHT READ TRAFFIC BY GROUP

Weights	Size	Reads per second	Bandwidth
FARGAN signal network	728 KB	400	284.4 MB/s
Acoustic stage	397 KB	100	38.8 MB/s

Weights	Size	Reads per second	Bandwidth
Prosody	196 KB	100	19.2 MB/s
FARGAN conditioning network	64 KB	100	6.2 MB/s

The blob is therefore emitted in two halves, so that the hot portion requiring 284 MB/s can be placed independently of the cold portion requiring an order of magnitude less. We verified the split against the single-blob path across a complete 2.34 second utterance of 37,440 samples, with none differing.

Table V is also the first indication of the finding that Section X establishes on hardware. The bandwidth figures are large compared with what a microcontroller flash interface supplies, and no arithmetic optimisation reduces them.

V. A WEIGHTLESS LPC VOCODER

A. Structure

The vocoder consumes the same 20-dimensional feature vector. It converts the 18 cepstral coefficients to an autocorrelation sequence, solves for linear prediction coefficients by Levinson recursion, and drives the resulting synthesis filter with a pulse train at the predicted pitch period, noise, or a mixture determined by the pitch correlation. It holds no learned parameters, occupies 5012 bytes of text and 1880 bytes of state, and costs 2.01 MMAC per second of audio.

B. Measured Improvements

A bare implementation of that description scores 2.414 SCOREQ against FARGAN's 3.744 on the same features. Four measured improvements close most of the gap, and each was tuned by sweep rather than by textbook value.

An adaptive postfilter emphasising formant peaks. The textbook narrowband coefficients proved wrong for 16 kHz material and the measured optimum is a numerator factor of 0.5 against a denominator factor of 0.8.

Source spectral shaping, guided by comparing the long-term average spectrum of the synthesiser against FARGAN's, which showed a deficit above 3 kHz that a first-order pre-emphasis on the excitation corrects.

Pulse dispersion in the style of MELP, since an all-pole filter driven by impulses is maximally peaky in time and real glottal excitation is not. A fixed all-pass dispersion filter spreads each pulse without altering its magnitude spectrum.

A band-split excitation with a voicing-dependent crossover, in which the excitation is periodic below the crossover and noisy above it. The measured optimum crossover sits far below the conventional 2 kHz.

Table VI gives the cumulative effect.

TABLE VI
CUMULATIVE EFFECT OF THE LPC IMPROVEMENTS

Configuration	SCOREQ
Bare LPC	2.414
Plus adaptive postfilter	2.628
Plus source shaping	2.752
Plus pulse dispersion	2.882
Plus band-split excitation	3.114
Plus word identifier repair	3.172
FARGAN on the same features	3.744

C. Result

The final entry in that table is not a vocoder change. Instrumenting the intermediate representation revealed that per-frame word identifier assignment collapsed every word in a phrase onto the identifier of the first, so a 117,456-parameter word embedding had been receiving a near-constant input throughout training and inference. Repairing the assignment is worth 0.058 SCOREQ and, more usefully, restores the pitch variation the embedding was supposed to supply.

The four vocoder improvements reach 3.114 SCOREQ, 83.2 percent of FARGAN's score, for 0.7 percent of its arithmetic and no weights. Repairing the word-identifier assignment raises the complete pipeline to 3.172 SCOREQ. The LPC synthesizer is also not a neural vocoder, so the deployed system is a neural acoustic pipeline with a classical vocoder rather than a neural graph end to end, and Section VII treats the consequences of that for comparison against published work.

VI. PRONUNCIATION DICTIONARY STORAGE

A. Repacking

CMUdict as a generated C structure occupies 5085 KB, which fits no target in this class. Measuring where that goes is instructive, since 1900 KB is index and pointers, roughly 700 KB is symbol table and alignment padding, and only 933 KB is the word characters themselves. Most of the cost is the representation rather than the data.

Repacking into a front-coded bucketed blob, in which words are sorted, grouped into buckets, and each entry stores only the characters that differ from its predecessor, gives the result in Table VII.

TABLE VII
DICTIONARY STORAGE BEFORE AND AFTER REPACKING

	Generated C table	Packed blob
Index and pointers	1900 KB	31 KB
Word characters	933 KB	330 KB
Phones and stress	1580 KB	791 KB
Record headers	0	244 KB
Symbol table and padding	700 KB	0
Total	5085 KB	1373 KB

All 124,912 entries were verified identical after repacking. The format costs approximately twelve bytes per word at any size,

so a restricted vocabulary scales down proportionally, and a 915-word embedded-systems vocabulary occupies 8.7 KB.

B. Access Pattern

The access pattern suits external storage. A binary search over the bucket index, held in 31 KB of RAM, locates the bucket without touching the card, leaving one read of a few hundred bytes per word. At forty to fifty words per second of speech that is a few kilobytes per second, which any SPI card supplies comfortably. The dictionary is therefore the one large item in the system genuinely suited to a card, and Section IX places it there.

VII. EVALUATION METHODOLOGY

A. Why Not PESQ

The original evaluation used PESQ throughout, and we established that it is unsuitable for this comparison by direct measurement. Two builds of an identical network, audibly indistinguishable, score 4.083 against each other rather than the 4.644 an identical file scores. Any PESQ figure in this domain must be read against that anchor, and differences smaller than half a point carry no information.

B. Metric Suite

We therefore report SCOREQ [10] and DNSMOS [11], which are no-reference, alongside a reimplement of ViSQOL speech mode [13], which is not. The distinction governs how each is used. No-reference metrics can score synthesised speech directly, which matters because there is no ground truth recording for text an acoustic model invented. ViSQOL is full-reference and requires a reference signal, so we use it only for build-to-build comparison, taking one configuration's output as the reference and another's as the test. The official ViSQOL binary requires Bazel and pulls in TensorFlow, so we reimplemented it, matching the upstream conformance constants exactly on the perfect-score case and within 0.006 on the real one.

C. Comparison Against Published Work

Table VIII places the pipeline against the nearest published microcontroller deployment.

TABLE VIII

PUBLISHED MICROCONTROLLER TTS QUALITY AGAINST THIS WORK

System	Parameters	SCOREQ
Amy Pareto package [12]	1,454,284	4.13
This work, FARGAN pipeline	1,428,037	3.744
sanoTTS embedded c-line [12]	567,008	2.54
This work, LPC pipeline	601,740	3.172

Three differences make that comparison indicative rather than controlled, and they are the kind the same authors warn about when they note that published embedded speech results do not all use the same system boundary. Their figures are computed

over twenty-four held-out sentences and ours over three, and their own work documents a narrow test set overstating a model by 1.35 SCOREQ. Their audio is 22.05 kHz and ours is 16 kHz, and SCOREQ is not calibrated across sample rates. And their parameter count covers a duration model, an acoustic model and a learned decoder, where ours covers two stages because the vocoder has no weights.

VIII. NEGATIVE RESULTS

Three plausible improvements were implemented, measured and abandoned. We report them because a negative result obtained with an instrumented harness is more useful than an untested hypothesis.

A. Allophonic Rules

Allophonic flapping and intervocalic stop shortening, both linguistically correct for an American reader, degrade quality. The acoustic model's receptive field is plus or minus 34 frames, and a two to four frame segment between two high-energy vowels is smeared by its neighbours, so the model cannot produce a closure that narrow. This is the same capacity limit that Section II identifies from the other direction.

B. Jitter and Shimmer

Jitter and shimmer degrade quality even after repairing an integer truncation that had silently disabled them. Periodicity did move toward FARGAN's, from 0.7200 to 0.7147, so the mechanism functions, but every setting lost quality. FARGAN is less periodic than this synthesiser and it is not because it varies its pitch. Its variation lies in the shape of each individual cycle, which a pulse train cannot express because every cycle is identical, and moving identical cycles around in time does not help.

C. Partial Pulse Dispersion

Partial pulse dispersion does nothing at all, which is diagnostic, because mixing a dispersed pulse with the original reconstitutes the peak and confirms that the effect is entirely in the phase.

Together these localise the remaining quality gap to per-cycle waveform variation, and indicate harmonic-plus-noise excitation as the next step at an estimated 5 to 8 MMAC per second of audio.

IX. DEPLOYMENT ON THE CH32H417

All execution-cost figures above are derived from instruction counts under emulation. This section and the next report measurements on physical silicon.

A. Division of work

The division follows the memory rather than the compute. The V3F owns the card, the dictionary, the frontend and the digital-to-analogue converter, because those need peripherals and a filesystem. The V5F owns the prosody model, the acoustic model and the vocoder, because those need a large arena and the faster core. A sixteen-word mailbox in shared SRAM

carries the command, the frame count, four buffer pointers and the status. The split costs almost nothing, since frontend and dictionary work is milliseconds against seconds of inference, and the benefit is that inference runs at twice the clock.

B. Flash banking and weight provisioning

The 960 KB of flash is two 480 KB banks with the boundary at 0x00078000. Both are readable and writable at runtime, but an executable image may not live in or cross bank 1. Images linked at the boundary and images spanning both banks both built and flashed, and neither produced output.

The 715 KB of weights therefore cannot be linked into an image, and are provisioned as data instead. On first boot the V3F reads three blobs from the card and programs them into flash at fixed addresses, verifying each 4096-byte chunk as it goes, then writes an 8 KB manifest holding the address, length and CRC32 of each. Every subsequent boot recomputes the three checksums from flash, and a match skips provisioning. Programming through the vendor fast page path measured 237 KB/s, so a first boot costs 4.1 seconds and nothing thereafter.

Three constraints determine the map. The acoustic weights at 397 KB fit only in bank 1. The prosody weights need 200 KB of erase-aligned space that crosses neither the bank boundary nor either image. And the V5F image address is fixed, for the reason given below. Table IX gives the arrangement that satisfies all three.

TABLE IX
CH32H417 FLASH MAP

Address	Contents	Size
0x00000000	V3F code	56 KB
0x0000E000	Prosody weights	196 KB
0x00040000	V5F code	64 KB
0x00050000	Word vocabulary	109 KB
0x00078000	Acoustic weights, bank 1	397 KB
0x000DC000	Manifest	8 KB

The prosody erase span ends exactly at the V5F image, and the V3F code region is 56 KB rather than 64 precisely because prosody needs those eight kilobytes to obtain a contiguous run below the boundary. Compile-time assertions verify every alignment and every erase span, so an inconsistent map fails to build rather than corrupting an image.

C. Bring-up failures worth recording

Three problems consumed more time than the port, and all three produce silence rather than a diagnostic.

The V5F wake address is defined in three places that need not agree, and the project header is included after the compiler command line and silently wins. When the wake address does not match the image, the second core executes whatever occupies that address, which under this map is a blob of int8 weights, and a core executing weight data writes to arbitrary registers. Twice this left the part read-protected. The image address is therefore pinned at the vendor default and the blobs

placed around it, with a compile-time comparison that fails the build on a mismatch.

Programming two images in one debugger session is not reliable. The second command reports a read-protect status that is not set, after which the debug module returns all ones until a power cycle. Separate invocations work. Programming the second image also erases the first unless the second file is a concatenation of both, in which case the driver writes the first section, reports the write discontinued at the address gap and abandons the rest, which is what leaves the earlier image intact.

The toolchain defaults to soft float on a part whose both cores implement the F extension and whose startup code already enables it. Correcting the architecture and ABI is a two-line change, but it must be accompanied by deleting the object cache, which is keyed on source path rather than on compiler flags.

D. Measured output quality

The system renders typed text to audio, writing each utterance to the card as a 16 kHz WAV as well as playing it. Table X gives DNSMOS scores for five sentences rendered by the device.

TABLE X
DNSMOS SCORES FOR FIVE UTTERANCES RENDERED ON THE DEVICE

Phrase	SIG	BAK	OVRL
the temperature is thirty seven degrees	3.41	3.85	3.05
all systems nominal	3.51	3.13	2.75
the motor is running normally	3.19	3.15	2.46
she sells sea shells	3.21	3.40	2.72
six thirty six	3.28	3.62	2.85
Mean	3.32	3.43	2.77

Five sentences is a spot check rather than a test set, and DNSMOS was trained on noise-suppression output rather than synthesis. The sibilant behaviour is as expected, since the two fricative-heavy sentences place 9.4 and 17.7 percent of their energy above 6 kHz against 0.0 to 1.9 percent for the other three, so the pulse-and-noise excitation produces frication rather than swallowing it.

X. OPTIMISATIONS MEASURED ON SILICON

The first working build ran at a real-time factor of 85.3 and the final one at 1.24, through five changes. Two of them returned far less than Section III predicted, and a third, which Section III does not model at all, returned more than any of the others.

Timings are wall-clock, taken on the V3F from a 1 kHz tick derived from the external crystal. The worker core publishes a stage marker as it enters each of prosody, the acoustic model and the vocoder, and the V3F, which is otherwise blocked polling the mailbox, timestamps each transition. Stage

boundaries are therefore accurate to one poll iteration, which is under a microsecond against stages measured in seconds. Playback and card access are excluded. Reported figures are single runs, which is defensible here because the workload is deterministic and uninterrupted: repeated renders of the same utterance on the same build agreed within 5 ms on a 2914 ms total, or 0.2 percent, and no run differed by more than one tick on any individual stage.

All figures are milliseconds of compute per second of rendered audio, which is length-independent, with cycles per multiply-accumulate computed against the arithmetic costs in Section III and corrected for the chunking multiplier, which is 1.000 for a 58-frame utterance and 1.581 for a 234-frame one.

A. Enabling the floating-point unit and the second core

Correcting the architecture and ABI took the pipeline from 85348 to 20016 ms per second of audio, a factor of 4.3 where Section III predicts 32. Moving prosody and the acoustic model to the 300 MHz core took it to 11260, a factor of 1.78 where the clock ratio predicts 2. Both fall short of the arithmetic prediction, and consistently enough to be a signal.

B. Weight staging

Both shortfalls have one explanation. The acoustic model sweeps its entire 397 KB of weights once per frame, because the frame loop sits inside each layer, which at a 148-frame window is 59 MB of reads per window. Both convolutions store weights as output by input by tap and iterate over the input index, so consecutive reads stride by seven or three bytes and none is contiguous. Neither a faster multiply nor a faster core speeds up a flash fetch, which is the practical form of the bandwidth figures in Table V.

The remedy changes no arithmetic. The tap loop hoists outside the frame loop, which is safe because each output accumulates over taps additively and still in ascending order, so every output sample is unchanged. With the tap fixed, one tap of a convolution is a 128 by 128 block of 16 KB, small enough to copy into RAM transposed so the inner loop reads contiguously. Flash traffic per window falls from 59 MB to 391 KB, the acoustic stage from 8371 to 964 ms per second, and the cost from 62.6 to 7.2 cycles per multiply-accumulate. Roughly 55 of every 63 cycles had been the core waiting on flash.

Prosody has the same weight sweep but is recurrent, so its timestep loop cannot move. It does not need to, since every timestep re-reads the same matrices and copying the 201 KB blob into RAM once per utterance is the whole fix. The staging buffer for each stage is the other stage’s arena, which is idle when it is needed, so neither costs additional memory. Prosody went from 2784 to 171 ms per second, a factor of 16. Both changes are bit-identical to the unstaged code.

C. Activation quantisation and two operating points

With the memory traffic removed the arithmetic became the limit, and the substitution Section III describes became worth making. Quantising each activation vector to int8 with one symmetric scale and accumulating in int32 took the acoustic

stage from 1431 to 971 ms per second and from 6.8 to 4.6 cycles per multiply-accumulate.

Unlike the FARGAN case, this substitution is not bit-exact. The weights were already int8, so half the error budget was spent, and quantising the activations spends the rest through seven convolution layers. The quality difference is audible. The sanoTTS authors report the same tension from the other direction, shipping a float-activation build for the browser because their 2.27 M model falls below their correlation gate under int8 activations and stays above it in float. Two independently developed small distilled models give the same finding, that on models of this size the activation quantisation is where the quality goes rather than the weight quantisation. We therefore ship both as selectable operating points rather than choosing one, and default to quality, since the 1.47 factor moves between two points that both require the listener to wait.

D. Changes that produced nothing

Flash access configuration moved the acoustic stage by 2 ms per second in 8372, which is noise, and setting the access clock to the undivided core clock broke programming, which the provisioning verification caught. Staging the weights as float32 rather than int8, which pays the widening once per window instead of once per element per frame and is exact, moved it from 963 to 972. Unrolling the dot product by four with a single accumulator was equally without effect. Enlarging the chunk to reduce the padding multiplier is bounded by memory rather than arithmetic, since halving the multiplier needs a 185-frame window and a 285 KB arena against the 234 KB available.

E. Summary

Table XI collects every measured build.

Build	Prosody	Acoustic	Vocoder	Total	RTF
150 MHz, soft float	15343	66840	3166	85348	85.3x
150 MHz, hard float	3151	16865	—	20016	20.0x
300 MHz core	2786	8371	103	11260	11.3x
Plus acoustic staging	2784	964	103	3852	3.9x
Plus prosody staging	171	1431	103	1706	1.7x
Plus int8 activations	171	971	103	1245	1.2x

The vocoder is unchanged throughout at 103 ms per second and 15.4 cycles per multiply-accumulate, which is worth noting on its own. It holds no weights, and it is the only stage in the pipeline whose cost was always arithmetic.

Against sanoTTS, which reports 5.72 times real time on an FPU-less 160 MHz core for a graph of about 45 MMAC per second of audio, our 49.9 MMAC per second executes at 4.6 to 6.6 cycles per multiply-accumulate against their 20.3. The

comparison is not like for like, since their pipeline is fully integer and ours retains a float accumulator in the quality mode, and their bottleneck was floating-point support code where ours was flash bandwidth. Both conclusions are correct for their respective parts and neither generalises to the other.

XI. LIMITATIONS

Silicon measurement covers one part and one configuration. No FARGAN configuration has run on hardware, and the CH32H417 measurements cover the LPC pipeline alone. Sections X.A and X.B also show that instruction counts overpredicted two changes by factors of 7 and 1.1 because they cannot see memory traffic, so the remaining emulated figures should be read as upper bounds on achievable speed rather than as predictions.

The evaluation set is small. SCOREQ figures are computed over three sentences and the on-device DNSMOS figures over five, so they establish the ordering of configurations that differ by a large margin and support no claim about differences smaller than that. The cumulative improvements in Table VI span 0.758 SCOREQ and the FARGAN gap is 0.572, both well outside what a three-sentence set can confuse; the 0.058 attributed to the word identifier repair is not, and is reported as a development measurement. The comparison in Table VIII is offered as an indication of scale rather than as a controlled result, for the reasons given in Section VII.C. Extending to a held-out set of the size used by the work we compare against is the next step and would change the strength of the claims rather than their direction.

Dimensions 12 to 17 at correlation 0.38 to 0.58 remain the principal acoustic bottleneck, and Section VIII shows that the receptive field also prevents the model from rendering short segments correctly. These are the same defect seen from two directions.

Text normalisation is unrepaired, real-time FARGAN is not reachable on this target without retraining at reduced width, and the system is single speaker, since LJSpeech is the only training corpus and multi-speaker extension requires speaker conditioning and a larger prosody model.

XII. CONCLUSION

We have taken a two-stage neural TTS pipeline from a research artefact to a deployed configuration on a RISC-V microcontroller, and measured rather than estimated at every step. The LPC configuration runs on a physical CH32H417 at 1.24 times real time in its fast mode and 1.71 in its quality mode, with a DNSMOS SIG of 3.32 measured on five sentences rendered by the device.

The result that shapes the rest is that on a core without a floating-point unit, an int8-weight network with float32 activations costs 229.2 instructions per multiply-accumulate while the same network with an int32 accumulator costs 7.06, identical to a hardware-float implementation. That change is bit-exact, and it means a single numeric path serves both target classes optimally.

The silicon measurements qualify that finding rather than overturning it. The same analysis predicted 32 times for enabling the floating-point unit and delivered 4.3, and predicted two times for doubling the clock and delivered 1.78, because an instruction count cannot see a weight fetch. Reordering the loops so that each kernel tap is staged in RAM once per window rather than re-read once per frame was worth 8.7 times on the acoustic stage and 16 on prosody, is bit-identical, and is worth more than every arithmetic optimisation in this paper combined. The instruction-exact method remains the right way to reason about arithmetic, and it needs a memory-traffic term beside it before it predicts a runtime.

The weightless vocoder demonstrates that much of the quality gap between classical and neural synthesis is recoverable with classical signal processing, provided each step is measured. Four improvements take the vocoder from 2.414 to 3.114 SCOREQ, or 83.2 percent of FARGAN's score, for 0.7 percent of the arithmetic, no weights, and 5012 bytes of code. Repairing a word-identifier defect raises the complete pipeline to 3.172 SCOREQ.

A theme runs through all three results. A lookup table beats a rational polynomial, a classical vocoder reaches 83 percent of a neural one, and a loop reordering beats every arithmetic optimisation attempted. On this class of hardware the cheap option is repeatedly the better one, and the only way to know which is to measure.

REFERENCES

- [1] J.-M. Valin, A. Mustafa, and J. B  the, "Very Low Complexity Speech Synthesis Using Frameworkwise Autoregressive GAN (FARGAN) With Pitch Prediction," *IEEE Signal Processing Letters*, vol. 31, pp. 2115–2119, 2024, doi: 10.1109/LSP.2024.3440956.
- [2] J.-M. Valin, A. Mustafa, C. Montgomery, T. B. Terriberry, M. Klingbeil, P. Smaragdis, and A. Krishnaswamy, "Real-Time Packet Loss Concealment With Mixed Generative and Predictive Model," in *Proc. Interspeech*, 2022, pp. 570–574, doi: 10.21437/Interspeech.2022-903.
- [3] K. Ito and L. Johnson, "The LJ Speech Dataset," 2017. [Online]. Available: <https://keithito.com/LJ-Speech-Dataset/>
- [4] ITU-T Recommendation P.862, "Perceptual Evaluation of Speech Quality (PESQ): An Objective Method for End-to-End Speech Quality Assessment of Narrow-Band Telephone Networks and Speech Codecs," Feb. 2001.
- [5] R. Luo et al., "LightSpeech: Lightweight and Fast Text to Speech With Neural Architecture Search," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2021, pp. 5699–5703, doi: 10.1109/ICASSP39728.2021.9414403.
- [6] Don't Ask Software, *Software Automatic Mouth (SAM)*, 1982.
- [7] Digital Equipment Corporation, *DECtalk*, 1984.

- [8] J.-M. Valin, J. Bütthe, A. Mustafa, and M. Klingbeil, “DRED: Deep REDundancy Coding of Speech Using a Rate-Distortion-Optimized Variational Autoencoder,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 18, no. 8, pp. 1441–1447, 2024, doi: 10.1109/JSTSP.2024.3482972.
- [9] J.-M. Valin and J. Bütthe, “Deep Audio Redundancy (DRED) Extension for the Opus Codec,” *IETF Internet-Draft draft-ietf-mlcodec-opus-dred-07*, Aug. 2026.
- [10] A. Ragano, J. Skoglund, and A. Hines, “SCOREQ: Speech Quality Assessment With Contrastive Regression,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 105702–105729.
- [11] C. K. A. Reddy, V. Gopal, and R. Cutler, “DNSMOS P.835: A Non-Intrusive Perceptual Objective Speech Quality Metric to Evaluate Noise Suppressors,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 886–890, doi: 10.1109/ICASSP43922.2022.9746108.
- [12] A. Thapa, “sanoTTS: The Smallest Real-Time Neural TTS on a General-Purpose Microcontroller,” *arXiv preprint arXiv:2608.21378*, 2026.
- [13] M. Chinen, F. S. C. Lim, J. Skoglund, N. Gureev, F. O’Gorman, and A. Hines, “ViSQOL v3: An Open Source Production Ready Objective Speech and Audio Metric,” in *Proc. 12th Int. Conf. Quality of Multimedia Experience (QoMEX)*, 2020, pp. 1–6, doi: 10.1109/QoMEX48832.2020.9123150.